README: Automated Web Security Scanner

Topic 2: Programmed a Python script to audit sites for secure headers and CORS flaws.

Participant: Tanisha (GitHub: tsh471/automated-web-scanner)

Program: IBM SkillsBuild National Cybersecurity Bootcamp

```
# Automated Web Security Scanner: Secure Headers and CORS Flaws Auditor

[![Python Version](https://img.shields.io/badge/python-3.8%2B-blue.svg)]
(https://www.python.org/)
[![Security Standards](https://img.shields.io/badge/standards-
OWASP%20ASVS%20%7C%20W3C%20CORS-orange.svg)](https://owasp.org/)
[![License](https://img.shields.io/badge/license-MIT-green.svg)](LICENSE)
[![Build Status](https://img.shields.io/badge/build-passing-brightgreen.svg)]()

> **IBM SkillsBuild National Cybersecurity Bootcamp Project Submission**
> **Topic 2:** *Automated Web Scanner: Programmed a Python script to audit sites for
secure headers and CORS flaws.*
> **Organized by:** BharatCares (CSRBOX Group) in collaboration with IBM SkillsBuild.

---

## 📌 Executive Summary

Web security misconfigurations—specifically missing HTTP security headers and overly
permissive Cross-Origin Resource Sharing (CORS) policies—represent some of the most
widespread vulnerabilities cataloged under **OWASP Top 10: A05:2021 (Security
Misconfiguration)**.

This repository contains an automated, multi-threaded Python security scanner developed
to:
1. Audit web endpoints against modern HTTP security header baselines.
2. Detect information disclosure and server fingerprinting leaks.
3. Actively probe endpoints for severe CORS misconfigurations, including arbitrary
origin reflection, `null` origin trust, and credential exposure.
4. Calculate a weighted **Security Score (0–100)** and letter grade (**A+ through F**).
5. Generate actionable reports in **Rich Terminal ANSI**, structured **JSON**, and
standalone **HTML**.

---

## 🚀 Key Features

- **Zero External Dependencies:** Built entirely with Python's standard library
(`urllib.request`, `concurrent.futures`, `json`, `argparse`, `dataclasses`, `ssl`). Runs
natively out of the box on any system with Python 3.8+.
- **Comprehensive Header Audits:**
```

```
  - `Content-Security-Policy` (CSP): Checks presence, weak directives (`'unsafe-inline'`, `'unsafe-eval'`), missing fallback directives (`default-src`, `script-src`), and plugin restrictions (`object-src 'none'`).
  - `Strict-Transport-Security` (HSTS): Validates HTTPS enforcement, `max-age` duration, `includeSubDomains`, and preload readiness.
  - `X-Frame-Options` (XFO) & CSP `frame-ancestors`: Evaluates protection against UI redressing and Clickjacking attacks.
  - `X-Content-Type-Options`: Enforces `nosniff` against MIME-sniffing exploits.
  - `Referrer-Policy`: Detects referral leakage policies (`strict-origin-when-cross-origin` vs. insecure `unsafe-url`).
  - `Permissions-Policy`: Audits browser API isolation (camera, microphone, geolocation).
  - Cross-Origin Isolation (`COOP`, `COEP`, `CORP`): Verifies process isolation defenses against side-channel attacks like Spectre.
  - Server Fingerprint Detection: Flags sensitive version disclosures (`Server`, `X-Powered-By`, `X-AspNet-Version`).
- **Active CORS Flaw Probing:**
  - **Arbitrary Origin Reflection:** Tests whether untrusted origins (e.g., `https://attacker-security-test.com`) are dynamically echoed into `Access-Control-Allow-Origin`.
  - **Credential Leakage (CRITICAL):** Flags when arbitrary origin reflection or wildcards are paired with `Access-Control-Allow-Credentials: true`, allowing attackers to steal session data via cross-origin JavaScript requests.
  - **Null Origin Trust:** Identifies endpoints trusting `Origin: null`, vulnerable to exploits via sandboxed `<iframe>` tags or `data:` URIs.
  - **Regex Subdomain / Prefix / Suffix Bypasses:** Tests for loose origin validation patterns (e.g., `target.com.attacker.com` or `attacker-target.com`).
  - **Insecure Protocol Trust:** Detects if HTTPS services permit cleartext `http://` origins (MITM exposure).
- **Flexible Reporting:**
  - Colored ANSI console output with tabular findings and direct remediation snippets.
  - Machine-readable JSON output for DevSecOps and CI/CD pipelines.
  - Standalone, publication-ready HTML reports with summary dashboard cards.
- **High-Performance Multi-Threading:** Scans dozens of URLs concurrently using `ThreadPoolExecutor`.
- **Offline Demo Mode (`--demo`):** Generates realistic simulated scans comparing vulnerable vs. hardened targets for academic grading and presentation.

---

## 🏗️ Architecture & Methodology

```
+---+
| TARGET URL(s) |
+---+
                                    |
                                    v
+---+
| WebScanner Orchestrator |
| (ThreadPoolExecutor Concurrent Workers) |
+---+
         | |
```
```

```
         v                                                  v
+----------------------------------+   +----------------------------------+
|      [1] HeaderAuditor           |   |       [2] CORSAuditor            |
| - Inspects initial HTTP response |   | - Sends active probe requests    |
| - Validates CSP, HSTS, XFO, XCTO |   | - Checks Arbitrary Origin Refl.  |
| - Detects information leakage    |   | - Tests Null Origin & ACAC combo |
| - Calculates score deductions    |   | - Tests Regex prefix/suffix trust|
+----------------------------------+   +----------------------------------+
         \                                                  /
          \                                                /
           v                                              v
+-------------------------------------------------------------------------+
|                         Scoring & Grading Engine                        |
|             (Base: 100 | Deductions for High/Medium/Low Risks)          |
+-------------------------------------------------------------------------+
                                    |
                                    v
+-------------------------------------------------------------------------+
|                             Reporter Engine                             |
|       - Console (ANSI)   |   - JSON Export   |   - Standalone HTML      |
+-------------------------------------------------------------------------+
```

📦 Installation & Setup

1. **Clone the repository:**
   ```bash
   git clone https://github.com/your-username/automated-web-security-scanner.git
   cd automated-web-security-scanner
   ```

2. **Verify Python version (Python 3.8+ recommended):**
   ```bash
   python3 --version
   ```

3. **Check Dependencies:**
   No external packages required! The scanner runs purely on Python's built-in standard library.

💻 Usage Guide

1. Single Target Scan
```bash
python3 scanner.py --url https://example.com
```

2. Multi-Target Batch Scan
Create a `targets.txt` file containing domains:

```text
https://example.com
https://google.com
https://github.com
```
Run with concurrency:
```bash
python3 scanner.py --file targets.txt --threads 8
```

3. Generate HTML Security Report
```bash
python3 scanner.py --url https://example.com --format html --output audit_report.html
```

4. Generate JSON Report for CI/CD Integration
```bash
python3 scanner.py --url https://example.com --format json --output audit_report.json
```

5. Run Offline Demonstration Mode
Ideal for evaluation, offline demonstrations, and project grading:
```bash
python3 scanner.py --demo
```

📊 Sample Terminal Output

```
================================================================================
     AUTOMATED WEB SCANNER: SECURE HEADERS & CORS FLAWS AUDITOR
                 IBM SkillsBuild Cybersecurity Bootcamp
================================================================================

--------------------------------------------------------------------------------
[1/1] Target: https://vulnerable-portal.local/api
Scan Time:     2026-10-24 15:46:57 | HTTP Status: 200 | Latency: 142.5 ms
Security Score: 35/100 (Grade: F)
--------------------------------------------------------------------------------

--- [1] HTTP SECURE HEADERS AUDIT ---
  [FAIL] Content-Security-Policy (Severity: HIGH)
         Missing CSP header. The application is vulnerable to Cross-Site Scripting
(XSS).
         Fix: Implement: default-src 'self'; script-src 'self'; object-src 'none';
  [FAIL] Strict-Transport-Security (Severity: HIGH)
         Missing HSTS header on HTTPS connection. Vulnerable to SSL-stripping downgrade
attacks.
         Fix: Add: Strict-Transport-Security: max-age=31536000; includeSubDomains;
preload
  [FAIL] X-Frame-Options (Severity: MEDIUM)
```

```
         Missing X-Frame-Options. Page can be framed for Clickjacking attacks.
         Fix: Set: X-Frame-Options: DENY
  [PASS] X-Content-Type-Options (Severity: PASS)
         MIME-sniffing protection is properly enabled ('nosniff').

--- [2] CORS MISCONFIGURATION PROBING ---
  [CRITICAL] Arbitrary Origin Reflection with Credentials (CRITICAL)
         Test Origin: https://attacker-security-test.com
         Finding:     Server reflects arbitrary Origin AND sets Access-Control-Allow-
Credentials: true.
         ACAO Header: https://attacker-security-test.com | ACAC Header: true
         Remediation: Implement strict server-side whitelist matching against trusted
domains.
======================================================================
```

🧪 Automated Testing

Run the included unit test suite:
```bash
python3 -m unittest discover -s tests -p "test_*.py" -v
```

All test cases validate:
- Accurate baseline header evaluation (Missing headers vs. Compliant headers).
- Loose CSP detection (`unsafe-inline`, `unsafe-eval`).
- Information leakage detection (`Server`, `X-Powered-By`).
- Security scoring and grade mapping accuracy.

📂 Project Structure

```
├── scanner.py          # Standalone CLI Scanner Engine (Header & CORS audit)
├── requirements.txt    # Specification of dependencies (zero runtime requirements)
├── README.md           # Project documentation and usage guide
├── sample_report.html  # Exported HTML audit report demo
├── sample_report.json  # Exported JSON audit report demo
└── tests/
    └── test_scanner.py # Automated unit test suite
```

🛡️ Remediation Reference Table

Header / Vector	Recommended Value	Risk Mitigated
Content-Security-Policy	`default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none';`	Cross-Site Scripting (XSS), Clickjacking

```
| **Strict-Transport-Security** | `max-age=31536000; includeSubDomains; preload` | SSL-stripping, MITM Downgrades |
| **X-Frame-Options** | `DENY` or `SAMEORIGIN` | UI Redressing / Clickjacking |
| **X-Content-Type-Options** | `nosniff` | MIME-Sniffing Exploits |
| **Referrer-Policy** | `strict-origin-when-cross-origin` | Sensitive Path / Parameter Leakage |
| **Permissions-Policy** | `camera=(), microphone=(), geolocation=()` | Unauthorized Hardware / API Access |
| **CORS Policy** | Explicit domain whitelist (No reflection with credentials) | Cross-Origin Data Exfiltration |

---

## 📜 Acknowledgements & Disclaimer

- Developed as part of the **IBM SkillsBuild National Cybersecurity Bootcamp** by **BharatCares (CSRBOX Group)**.
- **Ethical Use Notice:** This tool is intended exclusively for authorized security auditing, defensive hardening, and educational research. Unauthorized testing against third-party systems without permission is strictly prohibited.
```